

Intelligent Demand Forecasting Platform

A comprehensive architecture integrating climate-aware utility forecasting, industry-wise demand-supply clustering, real-time data streaming, and CNN+LSTM deep learning within a production-grade MLOps framework.

Prepared for Client Review
Version 1.0

Table of Contents

1. Executive Summary	2
2. Climate-Aware Civilian Utility Demand Forecasting	2
2.1 Key Climate & Utility Parameters	2
2.2 Forecast Horizons	3
2.3 Industry Benchmarks	3
2.4 Recommended Model Ensemble	3
3. Industry-Wise Clustering & BI Application with Streaming Data	4
3.1 Clustering Methodology	4
Pareto Segmentation (Tier 1)	4
Demand Pattern Clustering (Tier 2)	4
3.2 Streaming Data Architecture	4
3.3 BI Application Design	4
3.4 Capacity Orchestration	5
4. CNN+LSTM Deep Learning & MLOps Pipeline	6
4.1 CNN-LSTM Architecture	6
4.2 Performance Benchmarks	6
4.3 Classification Use Cases	6
4.4 MLOps Pipeline	7
4.5 Retraining Strategy	7
5. High-Level System Architecture	8
6. Technology Stack Summary	8
7. Implementation Roadmap	9
8. Appendix: CNN-BiLSTM Architecture Reference	10

1. Executive Summary

This document presents the technical architecture and client requirements for an **Intelligent Demand Forecasting Platform** designed to serve civilian utilities, industrial supply chains, and enterprise capacity planning. The platform integrates climate-aware forecasting, industry-wise demand-supply clustering, real-time data streaming, and deep learning models (CNN+LSTM) within a comprehensive MLOps framework.

Drawing on market-leading practices from PJM Interconnection, United Utilities, Sydney Water, HP Inc., and automotive supply chains across 90+ countries, this architecture delivers production-grade forecasting with measured accuracy improvements of 30-35% over traditional statistical methods.

3	4-6%	20 Weeks	CNN+LS TM
Requirement Areas	Target MAPE Accuracy	Implementation Timeline	Core Deep Learning Architecture

Figure 1: Platform at a glance

2. Climate-Aware Civilian Utility Demand Forecasting

Predict civilian demand for water, energy, and natural gas by integrating meteorological parameters as primary exogenous variables. This requirement addresses the critical dependency of utility consumption on weather patterns, seasonal cycles, and extreme climate events.

2.1 Key Climate & Utility Parameters

Parameter Category	Variables	Source	Granularity
Temperature	HDD, CDD, Min/Max/Mean temp	Met Office / NOAA API	Hourly/Daily
Precipitation	Rainfall, Snow, Drought indices	Weather Service	Daily
Seasonality	Spring/Fall flags, Daylight hours	Calendar-derived	Daily
Extreme Events	Heatwaves, Freeze-thaw, Storms	Weather Alerts	Event-based
Climate Projection	UKCP09/UKCP18 uplift factors	Climate Agencies	Annual
Calendar	Day of week, Holidays, Weekend flags	Internal	Daily
Utility Metering	Consumption (kWh, m3, liters)	Smart Meters / SCADA	15-min to Hourly

2.2 Forecast Horizons

Horizon	Duration	Use Case	Update Frequency
Short-term	24-72 hours	Operational dispatch, pump scheduling	Hourly
Medium-term	7-30 days	Tactical planning, maintenance windows	Daily
Long-term	1-5 years	Strategic capacity, infrastructure investment	Monthly/Quarterly

2.3 Industry Benchmarks

The table below summarizes production-grade implementations that inform our architecture:

Organization	Domain	Approach	Key Result
PJM Interconnection	US Energy Grid	HDD/CDD + sector-level models	Operational hourly load forecasting across residential, commercial, industrial
United Utilities (UK)	Water Resources	Met Office data + 1-in-500-year extreme scenarios	Demand forecasts extended to 2050 horizon
Welland Hydro (Canada)	Energy Distribution	Multivariate regression with HDD/CDD	R-squared = 88.9% for weather-normalized energy
Sydney Water (AU)	Water Distribution	Deep learning + weather integration	10-day operational short-term forecast framework
Tetouan/Astana Study	Energy (Research)	K-Means + Neural Network Regression	MAPE 5.19% vs. baseline 17.36%

2.4 Recommended Model Ensemble

- **Prophet** with external weather regressors for trend and seasonality decomposition
- **LightGBM** with lagged weather features (7-day, 30-day, 365-day lags) for gradient-boosted predictions
- **SARIMAX** with HDD/CDD covariates for statistical baseline
- **CNN-LSTM Hybrid** for deep sequence modeling with multivariate weather inputs
- **Ensemble blending** with horizon-aware weights optimized against wMAPE

3. Industry-Wise Clustering & BI Application with Streaming Data

Enable demand-supply optimization through industry-wise clustering of consumption patterns, real-time data streaming, and business intelligence dashboards designed for capacity orchestration across client operations.

3.1 Clustering Methodology

We implement a two-tier clustering strategy combining Pareto segmentation for high-value items with unsupervised learning for pattern-based grouping:

Pareto Segmentation (Tier 1)

- Top ~10% of items generating ~80% of revenue modeled individually
- Long-tail items pooled into clusters for shared model training
- Threshold determined by cumulative revenue curve analysis

Demand Pattern Clustering (Tier 2)

Algorithm	Strengths	Best For	Benchmark Score
K-Means	Fast, scalable, well-understood	Spherical clusters, initial segmentation	Silhouette: 0.724
DBSCAN	Handles outliers, arbitrary shapes	Noisy data, irregular demand patterns	Score: 0.89 vs K-Means 0.724
Hierarchical	Dendrogram visualization	Taxonomy creation, executive reporting	Interpretability: High

Recommendation: DBSCAN as primary clustering algorithm due to its robustness to noise and irregular cluster shapes, outperforming K-Means by 23% in retail benchmarks (score 0.89 vs. 0.724). K-Means retained for initial exploratory analysis.

3.2 Streaming Data Architecture

Component	Technology	Purpose
Message Broker	Apache Kafka / Redpanda	Real-time ingestion from IoT meters, POS, ERP
Stream Processing	Kafka Streams / Apache Flink	Windowed aggregations, real-time feature computation
Change Data Capture	Debezium	CDC from PostgreSQL/ERP source systems
Feature Computation	Feast (Feature Store)	Online feature serving for real-time inference
Anomaly Detection	Custom Spark Streaming jobs	Real-time anomaly scoring on demand patterns

3.3 BI Application Design

- **Interactive Dashboards:** MAPE/WMAPE tracking, forecast vs. actual comparison, bias decomposition by product category, region, and season
- **Revenue-Weighted Views:** Connect forecasting accuracy to business impact — over-forecast cost (excess inventory) vs. under-forecast cost (stockout/lost sales)
- **Role-Based Access:** Executive summary (KPIs), Planner workspace (drill-down forecast adjustment), Operator view (real-time alerting)
- **Alert System:** Automated notifications for forecast drift exceeding thresholds, anomaly detection triggers, and model performance degradation
- **What-If Scenario Engine:** Simulate weather events, promotion campaigns, supply disruptions, and capacity changes

3.4 Capacity Orchestration

- **Safety Stock Optimization:** Dynamic buffer calculation based on forecast uncertainty (prediction intervals)
- **Dynamic Reorder Points:** Adjust replenishment thresholds using real-time demand signals
- **Multi-Echelon Optimization:** Coordinate inventory levels across distribution centers, warehouses, and retail locations
- **Capacity Reservation:** Predictive booking of production slots, transport capacity, and storage based on forecast confidence

4. CNN+LSTM Deep Learning & MLOps Pipeline

Implement a CNN-LSTM hybrid architecture for demand classification and forecasting, integrated within a production-grade MLOps framework. The CNN layers extract local spatial and short-term temporal features, while bidirectional LSTM layers capture long-range dependencies in demand sequences.

4.1 CNN-LSTM Architecture

The proposed architecture follows the proven pattern from 2024 research showing CNN-LSTM hybrids outperform standalone models by 12-24% on MAPE:

Layer	Configuration	Purpose
Input	(batch, seq_len, n_features)	Multivariate time series window (30-90 days)
Conv1D	64 filters, kernel=3, ReLU, BatchNorm	Local pattern extraction (short-term trends)
Conv1D	128 filters, kernel=3, ReLU, BatchNorm	Higher-level feature extraction
MaxPool1D	pool_size=2	Dimensionality reduction, translation invariance
Dropout	Rate=0.3	Regularization, prevent overfitting
BiLSTM	128 units, return_sequences=True	Bidirectional temporal dependency modeling
BiLSTM	64 units, return_sequences=False	Final temporal encoding
Dropout	Rate=0.3	Regularization
Dense	32 units, ReLU	Feature compression
Dense (Output)	n_outputs	Regression: linear Classification: softmax

4.2 Performance Benchmarks

Based on published 2024 literature comparing models on supply chain demand forecasting tasks:

Model	MAE	RMSE	MAPE (%)	R-squared
ARIMA (Statistical Baseline)	143.67	180.24	15.29	0.62
Standalone LSTM	125.76	160.21	13.11	0.71
XGBoost	120.98	158.67	12.89	0.72
CNN-LSTM Hybrid	110.32	148.76	11.48	0.76
CNN-BiLSTM (2024 State-of-Art)	—	—	—	0.966 (Accuracy)

Key Finding: CNN-LSTM reduces MAPE by 25% vs. ARIMA and 11% vs. standalone LSTM. The 2024 CNN-BiLSTM variant achieves 96.57% accuracy on supply chain optimization tasks.

4.3 Classification Use Cases

Classification Task	Classes	Business Application
Demand Level	Low / Medium / High / Critical	Resource allocation, staffing levels
Anomaly Detection	Normal / Anomalous	Real-time alerting, fraud/leak detection
Demand Volatility Tier	Stable / Seasonal / Volatile / Intermittent	Model selection, safety stock policy
Event Impact	Normal / Holiday / Weather / Promotion	Decomposition, what-if analysis

4.4 MLOps Pipeline

A complete MLOps framework ensures reproducibility, monitoring, and continuous improvement:

Capability	Tool/Technology	Function
Experiment Tracking	MLflow	Artifact storage, model versioning, lineage tracking
Feature Store	Feast	Real-time feature serving, point-in-time correctness
Model Registry	MLflow Model Registry	Versioned models with metadata, stage transitions (Staging→Production→Archived)
CI/CD	GitHub Actions / GitLab CI	Automated testing, deployment, rollback
Orchestration	Apache Airflow / Prefect	Scheduled training pipelines, DAG management
Data Drift Monitoring	Evidently AI	PSI on input features, auto-trigger investigation
Concept Drift	Evidently AI / Custom	Rolling MAPE tracking, residual analysis
A/B Testing	Custom Champion/Challenger	Statistical validation of new model versions
Alerting	Prometheus + Grafana	Performance dashboards, Slack/Email alerts
Audit Trail	MLflow + Custom Logging	All predictions logged with model version and inputs

4.5 Retraining Strategy

- **Scheduled:** Monthly retraining with new data as default cadence
- **Performance-triggered:** When rolling MAPE exceeds baseline by >20%
- **Drift-triggered:** PSI > 0.25 on any key feature, or concept drift detected in residuals
- **Event-triggered:** Major market changes, new product launches, regulatory shifts

5. High-Level System Architecture

The platform follows a layered architecture with clear separation of concerns across data ingestion, feature engineering, modeling, serving, and MLOps layers:

DATA INGESTION LAYER
Weather APIs (NOAA, Met Office) Utility Smart Meters/SCADA Industry ERP/SCM Systems Streaming Platform (Kafka/MQTT for IoT and real-time feeds)
DATA LAKE (S3 / MinIO)
Raw data storage in columnar format (Parquet). Data versioning with DVC. Schema validation with Great Expectations.
FEATURE ENGINEERING LAYER
Lag features (t-1, t-7, t-30, t-365) Rolling statistics (7d/30d mean/std/min/max) Weather encodings (HDD, CDD, precip) Calendar features + cyclical encodings Cluster labels from DBSCAN/K-Means Holiday flags and event markers
MODELING LAYER (Ensemble)
Prophet (Trend + Seasonality) LightGBM (GBDT with lagged features) SARIMAX (Statistical with exogenous regressors) CNN-BiLSTM (Deep sequence modeling) Horizon-Aware Weighted Ensemble Blending
SERVING LAYER
FastAPI REST API (real-time inference) Streamlit Dashboard (BI/analytics) Power BI / Tableau Connector Alerting Service (Slack/Email/SMS) ERP/IBP Integration Connectors
MLOps LAYER
MLflow (Experiment Tracking + Model Registry) Feast (Feature Store) Evidently AI (Drift Detection) Apache Airflow (Pipeline Orchestration) GitHub Actions (CI/CD) Prometheus + Grafana (Monitoring)

Figure 2: Layered system architecture overview

6. Technology Stack Summary

Capability	Primary Technology	Alternative
Deep Learning	PyTorch / TensorFlow	JAX
ML Framework	LightGBM + Prophet + statsmodels	XGBoost + sktime
API Framework	FastAPI (Python 3.11+)	Flask, Django REST
Experiment Tracking	MLflow	Weights & Biases

Feature Store	Feast	Tecton
Data Versioning	DVC	Delta Lake
Orchestration	Apache Airflow	Prefect, Dagster
Streaming	Apache Kafka / Redpanda	AWS Kinesis
Monitoring	Evidently AI + Grafana	WhyLogs, NannyML
Containerization	Docker + Kubernetes	Podman
BI / Dashboard	Streamlit + Power BI	Tableau, Looker
Data Quality	Great Expectations	Soda, Deequ
CI/CD	GitHub Actions	GitLab CI, Jenkins

7. Implementation Roadmap

Phase	Duration	Key Deliverables
Phase 1: Foundation	Weeks 1-4	Data pipeline construction, weather API integration, basic Prophet & LightGBM models, baseline evaluation
Phase 2: Deep Learning	Weeks 5-8	CNN-LSTM implementation, hyperparameter tuning with Optuna, model evaluation against baselines, MLflow experiment tracking
Phase 3: Clustering & BI	Weeks 9-12	DBSCAN/K-Means industry clustering, Kafka streaming pipeline, Streamlit BI dashboards, what-if scenario engine
Phase 4: MLOps	Weeks 13-16	Feast feature store, Evidently drift monitoring, Airflow orchestration, CI/CD pipelines, automated retraining triggers
Phase 5: Production	Weeks 17-20	Load testing, security audit, A/B champion/challenger deployment, production cutover, user training & documentation

8. Appendix: CNN-BiLSTM Architecture Reference

The following is the reference PyTorch implementation for the CNN-BiLSTM hybrid model:

```
Input: (batch_size, sequence_length, n_features)
|
Conv1D(filters=64, kernel_size=3) -> ReLU -> BatchNorm
|
Conv1D(filters=128, kernel_size=3) -> ReLU -> BatchNorm
|
MaxPooling1D(pool_size=2)
|
Dropout(0.3)
|
BiLSTM(units=128, return_sequences=True)
|
BiLSTM(units=64, return_sequences=False)
|
Dropout(0.3)
|
Dense(32, activation=ReLU)
|
Dense(n_outputs) <- Regression: linear | Classification: softmax
```

Training Configuration: Adam optimizer with learning rate 0.001, ReduceLROnPlateau scheduler (factor=0.5, patience=10), EarlyStopping (patience=20), batch size 64, sequence length 60 days for short-term or 365 days for long-term forecasting, validation split 20% with TimeSeriesSplit cross-validation.